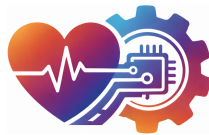




**UNIVERSIDAD
DE BURGOS**

Escuela Politécnica Superior

Grado en Ingeniería de la Salud



**TFG del Grado en Ingeniería de la
Salud**

**Desarrollo de un modelo
predictivo de glucemia a corto
plazo mediante técnicas de
aprendizaje automático en
pacientes con diabetes tipo 1
en terapia MDI**

Documentación Técnica

Presentado por Guillermo de la Fuente Abajo
en Universidad de Burgos

31 de mayo de 2026

Tutores: Antonio Jesús Canepa Oneto – Ruth Estefanía
Santos Mazo

Índice general

Índice general	1
Índice de figuras	3
Índice de tablas	4
Apéndice A Plan de Proyecto	5
A.1. Introducción	5
A.2. Metodología de ingeniería del software	6
A.3. Planificación temporal	6
A.4. Planificación económica	7
A.5. Viabilidad legal	10
Apéndice B Manual de especificación de diseño	13
Apéndice C Especificación de Requisitos	15
C.1. Diagrama de casos de uso	15
C.2. Casos de uso	16
C.3. Prototipos de interfaz	18
Apéndice D Documentación de usuario	21
D.1. Requisitos software y hardware para ejecutar el proyecto. . .	21
D.2. Instalación / Puesta en marcha	21
D.3. Manuales y/o Demostraciones prácticas	21
D.4. Requisitos software y hardware	21
D.5. D.2. Instalación y puesta en marcha	23

Apéndice E Manual del desarrollador / programador / investigador.	25
E.1. Estructura de directorios	25
E.2. Compilación, instalación y ejecución del proyecto	25
E.3. Pruebas del sistema	25
E.4. Instrucciones para la modificación o mejora del proyecto. . .	26
E.5. Estructura de directorios	26
E.6. Compilación, instalación y ejecución del proyecto	26
E.7. Pruebas del sistema	26
E.8. Instrucciones para la modificación o mejora del proyecto . .	27
Apéndice F Descripción de adquisición y tratamiento de datos	29
F.1. Descripción formal de los datos	29
F.2. Descripción clínica de los datos.	29
Apéndice G Estudio experimental	33
G.1. Cuaderno de trabajo.	33
G.2. Configuración y parametrización de las técnicas.	33
G.3. Detalle de resultados.	33
Apéndice H Anexo de sostenibilización curricular	35
H.1. Introducción	35
Apéndice I Registro de interacciones con sistemas de IA	37
I.1. Registro de interacciones con sistemas de IA	37
Bibliografía	39

Índice de figuras

A.1. Diagrama de Gantt del proyecto.	8
A.2. Fórmula para el cálculo de la amortización durante un periodo determinado.	8
C.1. Diagrama de casos de uso del sistema.	15
C.2. Interfaz gráfica del simulador py-mgipsim	18
C.3. Línea de comandos del simulador py-mgipsim	19
C.4. Visualización de serie glucémica	19
E.1. Diagrama del pipeline de análisis del proyecto.	28
F.1. Relación entre peso del vehículo y consumo (mpg) en mtcars . .	31

Índice de tablas

A.1. Milestones del proyecto y fechas de entrega.	7
C.1. CU-1 Ejecutar simulación de pacientes virtuales	16
C.2. CU-2 Visualizar series temporales de glucemia	17
C.3. CU-3 Exploración interactiva de datos por selección de paciente y escenario	18
D.1. Requisitos software del proyecto.	22
D.2. Especificaciones hardware del equipo de desarrollo utilizado como referencia.	22
F.1. Resumen estadístico del conjunto de datos mtcars	30

Apéndice A

Plan de Proyecto

A.1. Introducción

Este trabajo aborda el estudio comparativo de modelos predictivos de glucemia en pacientes con DM1 en régimen MDI. El alcance definitivo no estuvo definido desde el inicio: los primeros meses se dedicaron a explorar la viabilidad de acceder a datos reales de pacientes, lo que condicionó el arranque formal del proyecto.

A partir de octubre de 2025 se inició el contacto con la especialidad de endocrinología del Hospital Universitario de Burgos (HUBU) para identificar una cotutora clínica, valorar el tema y explorar el acceso a datos de pacientes reales. La reunión con el servicio hospitalario no pudo celebrarse hasta enero de 2026, momento en el que se constató que el acceso a datos reales presentaba barreras significativas: el proceso requería el consentimiento individualizado de cada paciente, los tiempos de respuesta dependían de las consultas clínicas y pocos pacientes disponían de registros completos de todas las variables de interés. Se exploró también la vía de la asociación local de diabetes (Asociación de Diabéticos de Burgos), con el mismo resultado. Paralelamente, desde enero de 2026 se inició la búsqueda de simuladores y datasets públicos como alternativa.

Una vez definida y aprobada la línea de trabajo, la planificación se formalizó mediante milestones e issues en [GitHub](#)¹ ([GitHub, Inc., 2026](#)), plataforma de gestión utilizada durante todo el desarrollo. No obstante, el trabajo previo de búsqueda de información, contacto con el hospital y

¹url repositorio: <https://github.com/WillyFtAb/TFG-GIS-Predictor-glucosa.git>

exploración de alternativas no quedó reflejado en GitHub ([GitHub, Inc., 2026](#)) al no haberse formalizado aún el proyecto.

A.2. Metodología de ingeniería del software

El trabajo siguió una metodología experimental iterativa, centrada en el análisis de datos y la evaluación de modelos más que en el desarrollo de un sistema software completo. Las fases se organizaron de forma secuencial pero con retroalimentación entre ellas: los resultados del preprocesamiento condicionaron decisiones de diseño experimental, y los resultados de la evaluación determinaron el alcance final de los modelos desarrollados.

La gestión de tareas se realizó mediante issues agrupadas en milestones en GitHub, lo que permitió un seguimiento incremental del progreso. Dado que el trabajo no incluye el desarrollo de un sistema software desplegable, no se aplicó ninguna metodología ágil formal.

Las fases previstas inicialmente que no pudieron completarse fueron: la validación con un segundo simulador o con datos reales, el análisis de significación de variables exógenas y la implementación de Prophet (?) como modelo adicional. Estas quedan identificadas como líneas futuras en el Capítulo 8 de la memoria.

De cara a un desarrollo software futuro basado en este trabajo —por ejemplo, un sistema de soporte a la decisión clínica para pacientes MDI— se recomienda una metodología ágil con sprints cortos (Scrum), dado que el dominio clínico requiere validación iterativa con especialistas y los requisitos pueden evolucionar con los resultados experimentales.

A.3. Planificación temporal

La planificación formal del proyecto se inició en marzo de 2026, una vez definida y aprobada la línea de trabajo. Se organizó en cinco milestones principales más un milestone de seguimiento para reuniones y tareas transversales. La Tabla [A.1](#) resume los hitos y la Figura [A.1](#) muestra su distribución temporal. A continuación se detallan las tareas asociadas a cada milestone.

Fase previa (oct. 2025 – ene. 2026): contacto con el servicio de endocrinología del hospital, búsqueda de cotutora clínica, exploración de acceso a datos reales y búsqueda de simuladores y datasets públicos. Esta fase no está reflejada en GitHub al ser anterior a la formalización del proyecto.

M1 — Obtención y validación de datos (ene. – mar. 2026): configuración del entorno y simuladores, diseño experimental, generación de datasets virtuales y validación clínica de los datos generados con la cotutora especialista.

M2 — Preprocesamiento de los datos (feb. – mar. 2026): implementación de un parser multiformato para la lectura de los archivos del simulador, resampling y ajuste temporal de las series a 5 minutos, y exploración de funciones de estandarización y normalización.

M3 — Desarrollo y entrenamiento del predictor (mar. – abr. 2026): implementación de XGBoost, implementación de ARIMAX, implementación y descarte de Prophet por incompatibilidad con el entorno, y ajuste de hiperparámetros mediante búsqueda bayesiana y grid search.

M4 — Validación clínica y métricas de error (abr. – may. 2026): diseño de la estrategia de evaluación, implementación del backtesting y visualización de resultados mediante métricas de error y Clarke Error Grid.

M5 — Escritura de memoria (may. – jun. 2026): redacción de todos los capítulos de la memoria, elaboración de anexos y revisión final para la entrega.

Seguimiento: reuniones periódicas con tutores a lo largo de todo el desarrollo, envío del compromiso alumno-tutor y redacción de la propuesta de TFG.

Tabla A.1: Milestones del proyecto y fechas de entrega.

Hito	Inicio	Fin previsto
Fase previa	Oct 2025	Ene 2026
M1: Obtención y validación de datos	Ene 2026	Mar 2026
M2: Preprocesamiento de datos	Feb 2026	Mar 2026
M3: Desarrollo y entrenamiento de modelos	Mar 2026	Abr 2026
M4: Validación clínica y métricas de error	Abr 2026	May 2026
M5: Escritura de memoria y entrega final	May 2026	Jun 2026

A.4. Planificación económica

La estimación económica del proyecto contempla los costes directos asociados al trabajo del alumno, la supervisión tutorial y el uso de equipamiento hardware. No se han incurrido costes de licencias de software, dado que

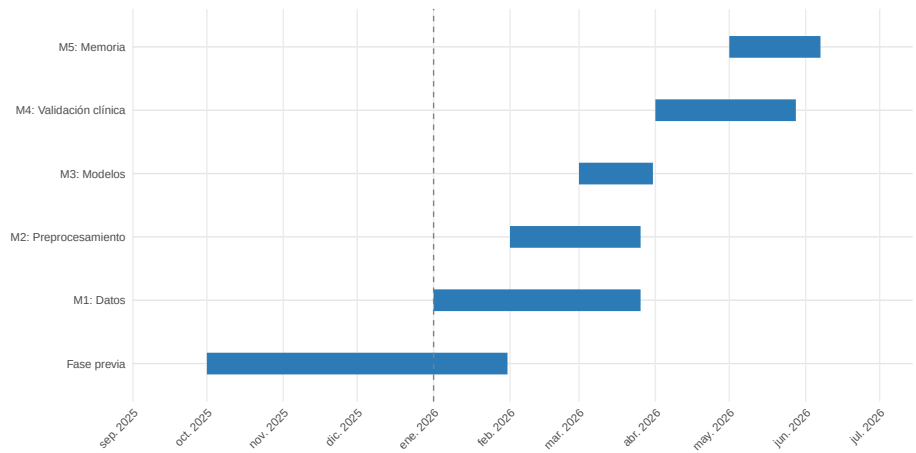


Figura A.1: Diagrama de Gantt del proyecto.

todas las herramientas utilizadas —Python ([Python Software Foundation, 2025](#)), R ([R Core Team, 2025](#)), Quarto ([Allaire, J.J. and Teague, Charles and Xie, Yihui and Dervieux, Christophe, 2022](#)), GitHub([GitHub, Inc., 2026](#)) y Google Colab ([Google, 2026](#)) en su modalidad gratuita— son de acceso libre.

Costes de personal

El coste del trabajo del alumno se estima a partir de las 360 horas previstas, aplicando el Salario Mínimo Interprofesional (SMI) vigente en 2026, fijado en 17.094 € brutos anuales por el Real Decreto 126/2026, lo que equivale a 7,10 €/hora para una jornada de 1.694 horas anuales (?). El coste de supervisión de los tutores se estima a partir de 20 horas totales de tutoría, aplicando una tarifa orientativa de técnico senior:

Concepto	Horas	Coste/hora (€)	Total (€)
Trabajo del alumno	360	7,10	2.556,00
Supervisión tutores	20	50,00	1000,00
Total personal			3.556,00

Costes de hardware

El desarrollo se realizó sobre hardware personal del alumno. La amortización de cada equipo durante el periodo de uso del proyecto se calcula según la expresión recogida en la figura [A.2](#).

$$\text{Amortización del periodo} = \frac{\text{Precio de compra}}{\text{Vida útil en años}} \times \frac{\text{Meses de uso}}{12}$$

Figura A.2: Fórmula para el cálculo de la amortización durante un periodo determinado.

Concepto	Coste (€)	Vida útil (años)	Meses de uso	Amortización (€)
Ordenador personal principal	[COMPLETAR]	5	6	[COMPLETAR]
Ordenador personal secundario	[COMPLETAR]	6	2	[COMPLETAR]
Total hardware				[COMPLETAR]

Consideraciones sobre el uso de hardware personal

El uso de ordenadores personales para el desarrollo del proyecto implica un coste temporal significativo derivado del tiempo de cómputo requerido por el backtesting y la optimización de hiperparámetros. Se estima un mínimo de 300 horas de uso intensivo, incluyendo pruebas y reinicios por fallos. Quien desee replicar el estudio debe contemplar este coste temporal como un factor limitante, especialmente si no se dispone de acceso a infraestructura de cómputo de alto rendimiento.

Costes de infraestructura cloud (referencia)

Aunque no se ha incurrido en coste alguno por uso de Google Colab ([Google, 2026](#)), se incluye como referencia el coste que habría supuesto el uso de la modalidad Colab Pro+ durante el periodo de desarrollo intensivo de modelos (aproximadamente 3 meses), a un precio de 49,99 \$/mes ([Google, 2026](#)):

Concepto	Coste mensual	Meses	Total
Google Colab Pro+	49,99 \$	3	149,97 \$

Resumen económico

Concepto	Total (€)
Personal	3.156,00
Hardware (amortización)	[COMPLETAR]
Software y licencias	0,00
Cloud (referencia, no incurrido)	~138,00
Total del proyecto	[COMPLETAR]

A.5. Viabilidad legal

El presente trabajo no ha requerido la aprobación de la Comisión de Bioética ni de ningún comité ético de investigación, dado que no involucra datos de pacientes reales. Todos los datos utilizados han sido generados mediante el simulador py-mgipsim (Siket et al., 2025), que implementa el modelo UVA/Padova (Man et al., 2014) sobre una cohorte de pacientes virtuales. Al tratarse exclusivamente de datos sintéticos, el trabajo queda fuera del ámbito de aplicación de la normativa de protección de datos de carácter personal.

Reglamento General de Protección de Datos (RGPD)

El Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos (Parlamento Europeo y Consejo de la Unión Europea, 2016), establece el marco normativo europeo en materia de privacidad. Su ámbito de aplicación material comprende el tratamiento total o parcialmente automatizado de datos personales, así como el tratamiento no automatizado de datos personales contenidos o destinados a ser incluidos en un fichero. Los datos de salud se clasifican como categoría especial de datos, cuyo tratamiento está prohibido con carácter general, salvo en supuestos tasados como el consentimiento explícito del interesado, la protección de intereses vitales o el interés público en el ámbito de la salud pública. Dado que el

presente trabajo no trata datos personales de ningún paciente real, el RGPD no resulta de aplicación.

Ley Orgánica 3/2018 de Protección de Datos Personales

La Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales (LOPDGDD) ([Jefatura del Estado, 2018](#)) adapta el RGPD al ordenamiento jurídico español y establece garantías adicionales para el tratamiento de datos sensibles. En materia de investigación sanitaria, la disposición adicional decimoséptima de la LOPDGDD introduce previsiones encaminadas a garantizar el adecuado desarrollo de la investigación en materia de salud, ponderando los beneficios que esta aporta a la sociedad con las debidas garantías del derecho fundamental a la protección de datos. Al no existir tratamiento de datos de pacientes reales en este trabajo, tampoco resulta de aplicación esta norma.

Licencias de software

Las herramientas de software utilizadas son de código abierto o de uso libre, sin restricciones de licencia que limiten su uso académico. El simulador py-mgipsim ([Siket et al., 2025](#)) se distribuye bajo licencia MIT ([Massachusetts Institute of Technology, 1988](#)), que permite su uso, modificación y distribución sin restricciones. Las librerías de Python ([Python Software Foundation, 2025](#)) y R ([R Core Team, 2025](#)) empleadas se distribuyen bajo licencias BSD ([Regents of the University of California, 1990](#)), Apache 2.0 ([Apache Software Foundation, 2004](#)) o GPL ([Free Software Foundation, 2007](#)), todas compatibles con el uso académico y la reproducibilidad del estudio.

Apéndice B

Manual de especificación de diseño

El presente trabajo no incluye desarrollo de hardware ni de un sistema software desplegable, por lo que los apartados de planos técnicos y diseño arquitectónico no resultan de aplicación.

Especificación de Requisitos

C.1. Diagrama de casos de uso

El trabajo no desarrolla un sistema software con interfaz de usuario final, por lo que el diagrama de casos de uso (C.1) se limita a las interacciones del investigador/alumno con las herramientas utilizadas durante el desarrollo. Se identifican tres casos de uso principales:

- **CU-1:** Ejecutar simulación de pacientes virtuales.
- **CU-2:** Visualizar series temporales de glucemia generadas.
- **CU-3:** Exploración interactiva de datos por selección de paciente y escenario.

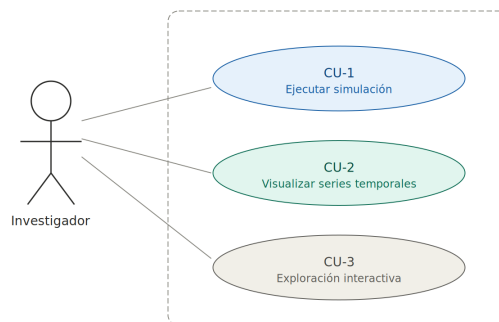


Figura C.1: Diagrama de casos de uso del sistema.

C.2. Casos de uso

CU-1	Ejecutar simulación de pacientes virtuales
Versión	1.0
Autor	Alumno
Descripción	El investigador ejecuta el simulador py-mgipsim para generar series temporales de glucemia bajo un escenario experimental definido.
Precondición	Python 3.12 instalado y dependencias del simulador instaladas mediante <i>pip install -r requirements.txt</i>
Acciones	El investigador selecciona la modalidad de ejecución (GUI o CLI). - Modalidad GUI: ejecutar <code>streamlit run interface_gui.py</code> en terminal, configurar los parámetros del escenario mediante la interfaz web y lanzar la simulación. - Modalidad CLI interactiva: ejecutar <code>interface_cmd.py</code>, cargar un escenario predefinido con <code>load -sn <nombre_escenario></code> y ejecutar <code>simulate</code>. - Modalidad CLI comando único: ejecutar <code>interface_cli.py [OPTIONS]</code> con los parámetros definidos en línea de comandos.
Postcondición	Los resultados se almacenan en <code>./SimulationResults/<carpetacondatettime>/. </code>
Excepciones	Versión de Python incompatible; dependencias no instaladas.
Importancia	Alta

Tabla C.1: CU-1 Ejecutar simulación de pacientes virtuales

CU-1	Ejecutar simulación de pacientes virtuales
Versión	1.0
Autor	Alumno
Descripción	El investigador ejecuta una función de visualización sobre los datos generados por el simulador para inspeccionar la serie glucémica de los paciente en un escenario determinado.
Precondición	Datos del simulador disponibles; entorno Python con matplotlib activo.
Acciones	1. El investigador abre el notebook de visualización. 2. Especifica el número de escenario y el identificador de paciente. 3. Ejecuta la función de visualización. 4. Se generan las gráfica con la serie temporal de glucemia de los paciente.
Postcondición	La gráfica se muestra en pantalla y puede exportarse como imagen
Excepciones	Fichero de datos no encontrado
Importancia	Media

Tabla C.2: CU-2 Visualizar series temporales de glucemia

CU-1	Exploración interactiva de datos por selección de paciente y escenario
Versión	1.0
Autor	Alumno
Descripción	El alumno selecciona por teclado el escenario y el paciente de interés para obtener una visualización directa de la serie glucémica, sin necesidad de modificar el notebook.
Precondición	Datos del simulador disponibles; entorno Python con matplotlib activo.
Acciones	1. El investigador ejecuta la función interactiva. 2. El sistema solicita por teclado el número de escenario (1–6) y el identificador de paciente (0–19). 3. El sistema valida la entrada. 4. Se genera y muestra la gráfica correspondiente.
Postcondición	La gráfica se muestra en pantalla.
Excepciones	Entrada no válida; datos del escenario no disponibles.
Importancia	Baja

Tabla C.3: CU-3 Exploración interactiva de datos por selección de paciente y escenario

C.3. Prototipos de interfaz

Las siguientes capturas muestran las dos modalidades de interacción con el simulador py-mgipsim ([Siket et al., 2025](#)) disponibles para el investigador.

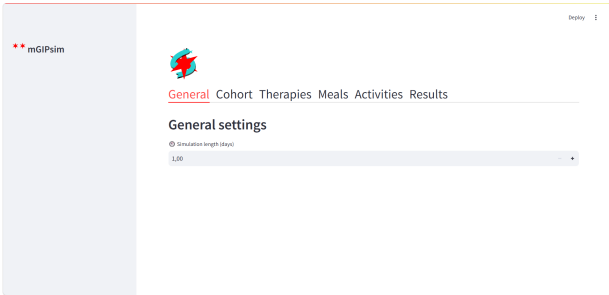


Figura C.2: Interfaz gráfica del simulador py-mgipsim

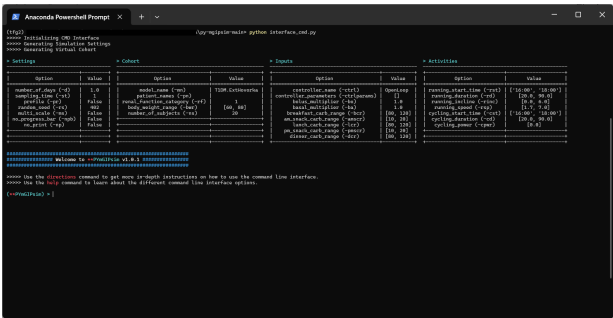


Figura C.3: Línea de comandos del simulador py-mgipsim

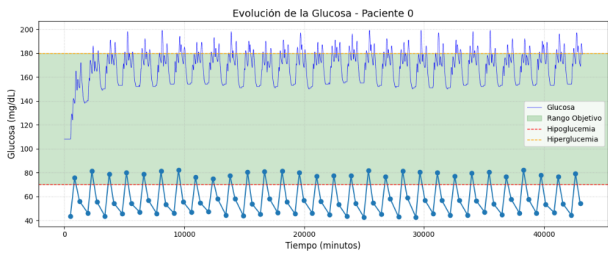


Figura C.4: Visualización de serie glucémica

Apéndice D

Documentación de usuario

D.1. Requisitos software y hardware para ejecutar el proyecto.

Se detallarán los requisitos mínimos de software y hardware necesarios para ejecutar correctamente el proyecto.

D.2. Instalación / Puesta en marcha

Se describirá el proceso de instalación y ejecución del sistema de forma clara y ordenada, indicando los pasos necesarios para su correcta puesta en funcionamiento.

D.3. Manuales y/o Demostraciones prácticas

Se incluirán instrucciones de uso del sistema y ejemplos prácticos que faciliten su comprensión por parte del usuario final.

D.4. Requisitos software y hardware

Requisitos software

Tabla D.1: Requisitos software del proyecto.

Software	Versión	Descripción
Python	3.12.0	Lenguaje de programación principal
R	4.x	Entorno para la compilación del documento
Quarto	1.x	Sistema de publicación del documento
conda	23.x o superior	Gestor de entornos y dependencias
Git	Reciente	Control de versiones
py-mgipsim	Última versión (descargada en 2026)	Simulador de pacientes virtuales

Las dependencias de Python del proyecto se recogen en el fichero `environment.yml` del repositorio, que permite reproducir el entorno de forma completa. Las versiones exactas de cada librería se especifican en dicho fichero.

Requisitos hardware

El proyecto no presenta requisitos estrictos de almacenamiento: los datos generados por el simulador ocupan menos de 10 GB en disco, una cantidad asumible para el estándar de almacenamiento actual.

El factor limitante es la capacidad de procesamiento y la memoria RAM disponibles. El backtesting con refit sobre 120 series temporales requiere tiempos de cómputo elevados; equipos con procesadores poco potentes o RAM insuficiente pueden hacer inviable la ejecución en tiempos razonables. El trabajo se ha desarrollado sobre un equipo con las siguientes especificaciones, que se proponen como referencia orientativa en la Tabla [D.2](#):

Tabla D.2: Especificaciones hardware del equipo de desarrollo utilizado como referencia.

Componente	Especificación
Procesador	AMD Ryzen 7 5700U (1,80 GHz, 8 núcleos)
RAM	16,0 GB (15,4 GB usable)

Componente	Especificación
Tarjeta gráfica	AMD Radeon Graphics (496 MB)
Sistema operativo	Windows 11

D.5. D.2. Instalación y puesta en marcha

Los pasos siguientes describen el proceso completo para reproducir el entorno y ejecutar el pipeline de análisis. Los pasos 3 y 4 (instalación y uso del simulador) son opcionales si se desea trabajar directamente con los datos ya generados incluidos en el repositorio.

Paso 1 — Clonar el repositorio

```
git clone https://github.com/WillyFtAb/TFG-GIS-Predictor-glucosa.git
cd TFG-GIS-Predictor-glucosa
```

Paso 2 — Crear el entorno conda e instalar dependencias

```
conda env create -f environment.yml
conda activate tfg-glucosa
```

Paso 3 — Instalar el simulador py-mgipsim (opcional)

```
git clone https://github.com/illinoistech-itm/py-mgipsim.git
cd py-mgipsim
pip install -r requirements.txt
```

Paso 4 — Generar nuevos datos con el simulador (opcional)

Una vez instalado el simulador, los datos pueden generarse mediante la interfaz gráfica o la interfaz de línea de comandos, tal como se describe en el Anexo C (CU-1). Los ficheros de resultados deben colocarse en la carpeta `data/` del repositorio para que el pipeline los localice correctamente.

Paso 5 — Visualizar los datos generados

Abrir y ejecutar el notebook de visualización:

```
jupyter notebook notebooks/visualizacion_datos.ipynb
```

Paso 6 — Ejecutar el pipeline de análisis

Abrir y ejecutar el notebook principal con las celdas en orden:

```
jupyter notebook notebooks/pipeline_prediccion.ipynb
```

Uso del notebook de visualización

El notebook de visualización permite inspeccionar las series temporales de glucemia generadas por el simulador. El usuario debe especificar en la celda de parámetros el número de escenario (1–6) y el identificador de paciente (0–19) para obtener la gráfica correspondiente. Las gráficas pueden exportarse como imagen desde la propia interfaz de Jupyter.

Uso del notebook de pipeline

El notebook de pipeline está organizado en secciones secuenciales que corresponden a las fases del análisis: preprocesamiento, ingeniería de características, optimización de hiperparámetros, backtesting y evaluación. Las celdas deben ejecutarse en orden. Los resultados se almacenan automáticamente en la carpeta **resultados/** del repositorio.

Los parámetros configurables por el usuario se encuentran en la celda de configuración al inicio del notebook, e incluyen la selección de escenarios a evaluar, los horizontes de predicción y las rutas de entrada y salida de datos.

Apéndice E

Manual del desarrollador / programador / investigador.

E.1. Estructura de directorios

Descripción de la estructura de carpetas y archivos entregados junto con el proyecto.

E.2. Compilación, instalación y ejecución del proyecto

En caso necesario, se detallarán los pasos técnicos para compilar, instalar y ejecutar el proyecto.

E.3. Pruebas del sistema

Esta sección puede ser opcional.

Se describirán las pruebas realizadas para validar el correcto funcionamiento del sistema, tales como pruebas unitarias, funcionales o validaciones con usuarios.

E.4. Instrucciones para la modificación o mejora del proyecto.

Se proporcionarán indicaciones y recomendaciones para facilitar futuras ampliaciones o mejoras del trabajo realizado.

E.5. Estructura de directorios

[COMPLETAR cuando la estructura del repositorio esté definida]

E.6. Compilación, instalación y ejecución del proyecto

El proceso de instalación y ejecución del entorno se describe en detalle en el Anexo D (apartado D.2). Desde el punto de vista del desarrollador, se señalan los siguientes aspectos técnicos adicionales:

El entorno de desarrollo se gestiona mediante conda. El fichero `environment.yml` del repositorio recoge todas las dependencias con sus versiones exactas, garantizando la reproducibilidad del entorno. Para activar el entorno:

```
conda env create -f environment.yml
conda activate tfg-glucosa
```

Las funciones de preprocesamiento, ingeniería de características y evaluación están implementadas en el módulo `[COMPLETAR].py`, importado por los notebooks principales. Cualquier modificación sobre estas funciones se propagará automáticamente a todos los notebooks que las utilicen, sin necesidad de duplicar código.

E.7. Pruebas del sistema

El proyecto no incorpora un sistema formal de pruebas unitarias. La validación del correcto funcionamiento de las funciones de preprocesamiento, ingeniería de características y backtesting se realizó de forma manual durante el desarrollo, mediante la visualización de las series temporales de glucemia,

la inspección de los dataframes intermedios y la comprobación de coherencia con los ficheros de origen (CSV y JSON) generados por el simulador.

El notebook de visualización (véase Anexo D, apartado D.3) constituye la herramienta principal para verificar visualmente que los datos preprocesados son correctos antes de ejecutar el pipeline de análisis. Las celdas de prueba con datos sintéticos utilizadas durante el desarrollo han sido eliminadas del pipeline final, que contiene únicamente el flujo de trabajo definitivo.

E.8. Instrucciones para la modificación o mejora del proyecto

El diagrama de la Figura E.1 ilustra el flujo completo del pipeline de análisis, desde la generación de datos hasta la evaluación de resultados.

Las principales extensiones y modificaciones que pueden realizarse sobre el proyecto son las siguientes:

Incorporación de nuevos modelos predictivos: para añadir un nuevo modelo al pipeline, es necesario implementar la función de entrenamiento y predicción siguiendo la interfaz de `skforecast` ([Amat Rodrigo and Escobar Ortiz, 2024](#)), definir la estrategia de búsqueda de hiperparámetros y añadir el modelo al bucle de `backtesting`. El resto del pipeline (preprocesamiento, evaluación y visualización de resultados) es reutilizable sin modificaciones.

Nuevos escenarios experimentales: la generación de nuevos escenarios se realiza directamente desde el simulador `py-mgipsim` ([Siket et al., 2025](#)), modificando los parámetros de variabilidad en la ingesta, ejercicio y error en la dosis. Los nuevos ficheros de salida deben colocarse en la carpeta de datos del repositorio siguiendo la convención de nomenclatura establecida.

Validación con datos reales: la sustitución de los datos simulados por datos reales requiere adaptar el parser de lectura al formato del dataset utilizado (por ejemplo, `OhioT1DM` ([Marling and Bunesu, 2020](#))) y verificar que las variables exógenas disponibles son compatibles con las esperadas por las funciones de ingeniería de características.

Análisis de explicabilidad: la integración de SHAP o de la extracción sistemática de importancia de características puede incorporarse al bucle de `backtesting` añadiendo una llamada a las funciones correspondientes tras cada paso de `refit`, almacenando los valores de importancia junto con las métricas de evaluación.

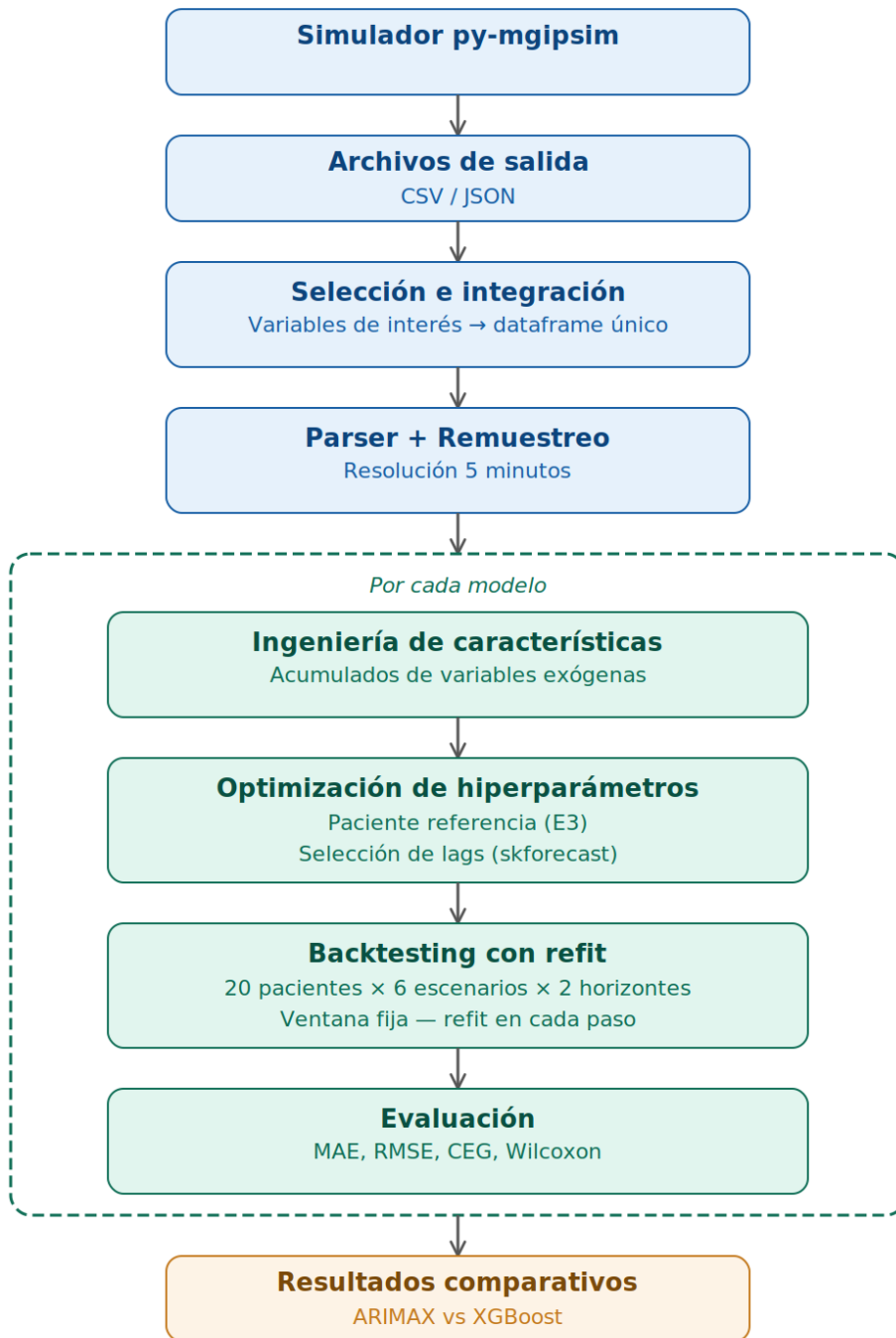


Figura E.1: Diagrama del pipeline de análisis del proyecto.

Apéndice *F*

Descripción de adquisición y tratamiento de datos

F.1. Descripción formal de los datos

Se describirá el tipo de datos utilizados, su formato, origen, volumen y características principales. Tablas, imágenes, señales, secuencias de ADN...

F.2. Descripción clínica de los datos.

Descripción y explicaciones clínicas del significado o interpretación de los datos.

A continuación se muestra un **ejemplo de análisis exploratorio con R** que produce tanto código visible como tabla y figura en el PDF:

```
library(knitr)
library(dplyr)

mtcars |>
  select(mpg, cyl, hp, wt) |>
  summarise(across(everything(),
    list(media = mean, sd = sd, min = min, max = max),
    .names = "{.col}_{.fn}"
  )) |>
  tidyr::pivot_longer(everything(),
    names_to = c("variable", "estadístico"),
```

Tabla F.1: Resumen estadístico del conjunto de datos mtcars

Variable	Media	SD	Mín	Máx
mpg	20.09	6.03	10.40	33.90
cyl	6.19	1.79	4.00	8.00
hp	146.69	68.56	52.00	335.00
wt	3.22	0.98	1.51	5.42

```
names_sep = "_"
) |>
tidyr::pivot_wider(names_from = estadístico, values_from = value) |>
mutate(across(where(is.numeric), \(x) round(x, 2))) |>
kable(booktabs = TRUE, format = "latex",
      col.names = c("Variable", "Media", "SD", "Mín", "Máx"))
```

```
library(ggplot2)

ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point(aes(color = factor(cyl)), size = 2.5) +
  geom_smooth(method = "lm", se = TRUE, color = "grey40") +
  labs(
    x = "Peso (1000 lbs)",
    y = "Consumo (mpg)",
    color = "Cilindros"
  ) +
  theme_minimal(base_size = 11)
```

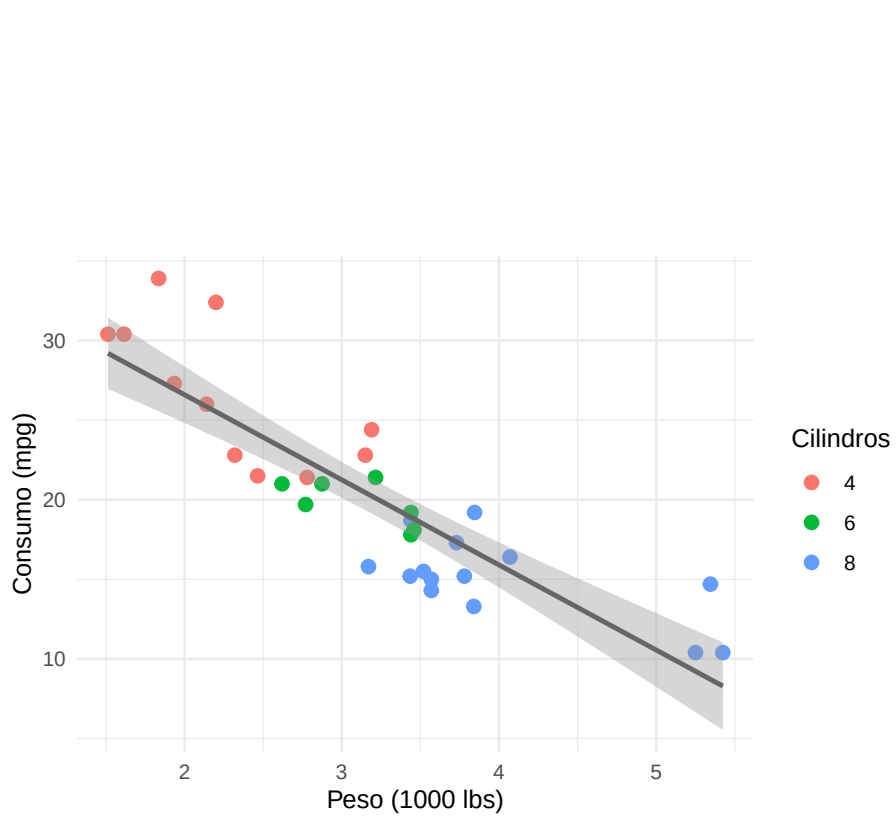



Figura F.1: Relación entre peso del vehículo y consumo (mpg) en mtcars

Apéndice G

Estudio experimental

G.1. Cuaderno de trabajo.

Se enumerarán los experimentos realizados durante el proyecto, incluyendo tanto los resultados satisfactorios como aquellos que no hayan sido exitosos.

G.2. Configuración y parametrización de las técnicas.

Se detallarán los parámetros utilizados en los experimentos y la justificación de su elección.

G.3. Detalle de resultados.

Se presentarán los resultados obtenidos mediante tablas, figuras y métricas relevantes.

Apéndice H

Anexo de sostenibilización curricular

H.1. Introducción

Este anexo incluirá una reflexión personal del alumnado sobre los aspectos de la sostenibilidad que se abordan en el trabajo. Se pueden incluir tantas subsecciones como sean necesarias con la intención de explicar las competencias de sostenibilidad adquiridas durante el alumnado y aplicadas al Trabajo de Fin de Grado.

Más información en el documento de la CRUE https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sostenibilidad_Crue2012.pdf.

Este anexo tendrá una extensión comprendida entre 600 y 800 palabras.

Apéndice I

Registro de interacciones con sistemas de IA

I.1. Registro de interacciones con sistemas de IA

Este anexo tiene como objetivo garantizar la transparencia en el uso de herramientas de inteligencia artificial generativa durante la realización del Trabajo de Fin de Grado. En él se deberá incluir un registro ordenado de los *prompts*, solicitudes, o peticiones formuladas por el estudiante en las interacciones con sistemas de IA que hayan sido más significativas en relación al desarrollo del TFG. Asimismo se incluirán bien las respuestas generadas directamente por los sistemas de IA, o bien se reflejará la importancia que ha tenido esa interacción en el desarrollo del trabajo (código, texto, enfoque...) de esas interacciones significativas.

El propósito de este anexo no es penalizar el uso de inteligencia artificial, sino documentarlo de forma clara, permitiendo al tribunal evaluar el alcance, la finalidad y el impacto real de dichas herramientas en el proceso de aprendizaje y en los resultados obtenidos. El uso de IA debe entenderse como una herramienta de apoyo, nunca como un sustituto del razonamiento, la toma de decisiones técnicas o la autoría del trabajo.

Para cada interacción relevante con sistemas de IA, se deberá indicar, al menos, la siguiente información:

- Herramienta o modelo de IA utilizado.
- Fecha aproximada de uso.

- Prompt/petición/solicitud/forma de uso realizada por el estudiante.
- Respuesta completa generada por la IA (opcional)
- Uso posterior de dicha respuesta en el TFG (orientación conceptual, mejora de redacción, generación de ideas, apoyo técnico, generación de código, etc.).

Este anexo podrá estructurarse en forma de tablas o listados numerados, y solo deberá incluir aquellas interacciones que hayan tenido una influencia significativa en el desarrollo del trabajo.

- perplexity
- 10/04
- @académico busca casos de utilización del algoritmo xgboost para la predicción de series temporales con multiples variables de entrada
- Descubrimiento de la librería *Skforecast*.

- perplexity
- 10/04
- @académico find scientific papers about the effect of past blood glucose levels, insulin bolus and exercise on future glucose value

Effects of postprandial exercise on blood glucose levels in adults with type 1 diabetes: a review of the literature (2023). Resume cómo el ejercicio después de comer cambia la glucosa futura y cómo las reducciones de insulina prandial antes del ejercicio modifican el riesgo de hipoglucemia.

The impact of brief high-intensity exercise on blood glucose levels (2013). Revisa estudios sobre ejercicio intenso breve y muestra efectos agudos y también a 1–3 días sobre sensibilidad a la insulina y glucosa.

Ejercicio y niveles de glucosa en la diabetes (ADA, 2024). Explica que el ejercicio suele bajar la glucosa, pero algunos tipos pueden subirla por adrenalina, y que el efecto puede durar horas o incluso hasta 48 horas.

- Utilización para el diseño y creación de *lags* en el dataframe de entrada para los algoritmos y base científica.

Bibliografía

- Allaire, J.J. and Teague, Charles and Xie, Yihui and Dervieux, Christophe (2022). Quarto.
- Amat Rodrigo, J. and Escobar Ortiz, J. (2024). skforecast: time series forecasting with scikit-learn.
- Apache Software Foundation (2004). Apache license, version 2.0.
- Free Software Foundation (2007). GNU general public license, version 3.
- GitHub, Inc. (2026). Github. Plataforma global de alojamiento de código fuente y desarrollo colaborativo.
- Google (2026). Google collaboratory. <https://colab.research.google.com/>.
- Jefatura del Estado (2018). Ley orgánica 3/2018, de 5 de diciembre, de protección de datos personales y garantía de los derechos digitales. Boletín Oficial del Estado.
- Man, C. D., Micheletto, F., Lv, D., Breton, M., Kovatchev, B., and Cobelli, C. (2014). The UVa/Padova type 1 diabetes simulator: new features. *Journal of Diabetes Science and Technology*, 8(1):26–34.
- Marling, C. and Bunescu, R. (2020). The ohiot1dm dataset for blood glucose level prediction: Update 2020. *CEUR workshop proceedings.*, 2675:71—74.
- Massachusetts Institute of Technology (1988). The MIT license.
- Parlamento Europeo y Consejo de la Unión Europea (2016). Reglamento (UE) 2016/679 del parlamento europeo y del consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales. Diario Oficial de la Unión Europea.

- Python Software Foundation (2025). Python language reference, version 3.
- R Core Team (2025). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria.
- Regents of the University of California (1990). The BSD 3-clause license.
- Siket, M., Rashid, M. M., and Cinar, A. (2025). py-mgipsim: An Open-Source Python Library for Simulating Type 1 Diabetes With Diverse Meals and Physical Activities. *Journal of Diabetes Science and Technology*, 19(3):855–856.